

Clasificacion de Imagenes de Comida

Proyecto final en equipo de vision por computadora



Integrantes: Lolita Alva Carreño, 21111241 y Daniel Octavio Ramirez Neri, 21111240

Entorno: Debian + NVIDIA GeForce RTX 4060

Problema y objetivo

- Problema: reconocer automaticamente el tipo de comida en una foto.
- Objetivo: clasificar cada imagen en **1 de 34 clases**.
- Producto final: una web simple que desarrollamos para subir imagen y ver el resultado.

Dataset

- Total de imagenes: **23,873**
- Numero de clases: **34**
- Split: 70% train, 15% val, 15% test



Preprocesamiento explicado facil

- Resize: todas las imagenes al mismo tamaño (224x224 en el modelo final).
- Normalizacion: ajustar valores de pixeles para entrenar de forma estable.
- Augmentation: crear variaciones (giro, zoom, color) para generalizar mejor.

Evolucion de modelos

- 1) Modelo base sin convoluciones fuertes: **21.16%**
- 2) ResNet18, una CNN: **78.78%**
- 3) ResNet50, una CNN: **91.23%**

Mensaje clave: al pasar a redes convolucionales, mejoramos mucho el rendimiento.

Que es ResNet y para que sirve?

- **ResNet** es una **CNN** (red neuronal convolucional) con conexiones residuales.
- Sirve para clasificacion de imagenes cuando hay muchos patrones visuales.
- Se usa en comida, medicina, industria, seguridad y vision para robots.
- Ventaja principal: permite entrenar redes mas profundas sin perder rendimiento.

Por que usamos ResNet y no una CNN "simple"?

CNN simple

- Mas facil de implementar
- Pero suele quedar corta en datasets complejos

ResNet, nuestra eleccion

- Tambien es CNN, pero mas profunda y robusta
- Mejor transfer learning con ImageNet
- Resultado final mucho mas alto: 91.23%

Filtros convolucionales usados

- No pusimos filtros manuales; ResNet los aprende durante el entrenamiento.
- Primera capa: **64 filtros de 7x7**.
- Bloques internos: convoluciones **1x1**, **3x3** y **1x1**.
- Con fine-tuning, adaptamos esos filtros a clases de comida.

Entrenamiento (resumen)

- Optimizador: AdamW
- Scheduler: ReduceLROnPlateau
- Control de sobreajuste: early stopping + dropout
- Seguridad del entrenamiento: checkpoint del mejor modelo

Resultados

Validation Accuracy: **91.01%**

Test Accuracy: **91.23%**

Salida en inferencia: clase, confianza y top-5

Aplicacion web (Flask)

- Frontend: selector de imagen, vista previa y resultado.
- Backend: endpoint `/predict` que carga el `.pth` y predice.
- Respuesta: clase predicha, confianza y top-5.

Codigo clave (inferencia)

- Usamos `torch.no_grad()` para desactivar gradientes y hacer la inferencia mas rapida.
- Aplicamos `softmax` para convertir la salida del modelo en probabilidades por clase.
- Con `probs.max(1)` obtenemos la clase ganadora y su confianza.
- Mapeamos el indice predicho a `CLASSES` para mostrar el nombre real de la comida.

```
with torch.no_grad():
    output = model(img_tensor)
    probs = torch.nn.functional.softmax(output, dim=1)
    conf, pred = probs.max(1)

pred_idx = pred.item()
clase = CLASSES[pred_idx]
confianza = conf.item() * 100
```

Conclusiones

- ResNet si es una CNN, y fue nuestra mejor opcion para este problema.
- La profundidad + transfer learning explican la mejora de 21% a 91.23%.
- El sistema ya funciona de extremo a extremo: modelo + app web.

Gracias

Preguntas?